classmate
Date
Page

* Unit-1 ⇒ 23%. Basics of Algo, Analysis of Algo.

- Algorithm, Properties, T & S complexity, Analysis of Algo
- Mathematics, fn & Relations, Vector & Matrices, Linear Ineq & Equns
- Efficient Algorithm, Asymptotic Notations (Best, Average, Worst)
- Amortized Analysis, Recursive Algo, Recurrence Relations,
- Sorting Algorithm: Bubble, Selection, Insertion, Shell, Heap
  Linear Time: Bucket, Radix, Counting

* Unit-2 ⇒ 15%. D & C Algo

- Solving Recurrence Relations
- Multiplying Large Integers Problem
- Binary Search, Max-Min Problem, Merge Sort, Quick Sort
- Matrix Multiplication.

* Unit-3 ⇒ DP & Greedy ⇒ 30%.

DP
- Optimality, Binomial coefficient, Making change,
- Assembly line scheduling, Knapsack, All points shortest path,
- Matrix Chain multiplication, Longest common subsequence.

Greedy
- Characteristics, Activity Selection, Elements of Greedy Strategy
- MST (Prims, Kruskals)
- Graphs & : Shortest Paths, Knapsack, Job Scheduling, Huffman Code

* Unit-4 ⇒ Graphs ⇒ 10%.

- Undirected, directed, Traversing, BFS, DFS, Topological sort

Units
Backtracking, Branch & Bound: 8 Queens, Knapsack, Travelling Salesman, Minimax principle — 10%.

classmate
Date
Page 1

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

16 September 2025 — Tuesday — 8:33 PM

Q1) Define algorithm. What are the essential properties of an algorithm? How do you compare two algorithms?

Ans1) An algorithm is a finite sequence of well defined, step-by-step instructions for solving a problem or performing a computation.

It takes some input, processes it through logical steps & produces an output.

It is the procedure for problem solving in the language of Mathematics & computations.

* Essential Properties:-

- The algorithm must always terminate after a finite number of steps. Infinite loops or never-ending processes are not valid algorithms.
- Each step must be definite, precisely & unambiguously defined.
- An algorithm should have zero or more well defined inputs. Inputs provide the initial data for the algorithm to work-on.
- It must produce at least one output.

✱ Comparing algorithms

- **Correctness**: Does the algorithm always produce the right output for all valid inputs?
- **Time complexity**: How much time does the algorithm takes as a function of input size (n)? Expressed in Asymptotic notations like Big-O, Θ, Ω.
- **Space complexity**: How much memory or additional storage does the algorithm require?
- **Scalability**: How well does the algorithm perform as the problem size grows very large? Some algorithms are efficient for ~~large~~ small data but impractical for large datasets.
- **Ease of implementation**: Is the algorithm easy to understand, implement, debug & maintain? Sometimes a slightly less efficient but simpler algorithm is preferred in practice.

✱ Trade-off often exists between time & space complexity.

→ An algorithm is a finite, precise, effective sequence of steps that maps inputs to outputs.

→ Essential properties: finiteness, definiteness, input, output, effectiveness.

→ Comparison: check correctness, time & space complexity, scalability, simplicity & robustness.

Q2) Asymptotic notations (Big-O, Θ, Ω). Why are they important in algorithm analysis?

Ans2) Asymptotic notations describe the growth rate of an algorithm's running time or space as the input size (n) becomes very large.

They ignore machine dependent constants & lower order terms to focus on the core efficiency of the algorithm. This helps us compare algorithms independently of these external factors.

1) Big-O Notation (O):

- It gives the upper bound of an algorithms growth rate.
- It means that the running time will not grow faster than this bound, for large n.
- It represents worst case performance.
- Example: Consider Linear Search, searching an element in a unsorted array of size n. In the worst case, we check all the elements.

  Time Complexity = O(n)

2) Big - Ω Notation (Ω):

- It gives the lower bound of an algorithm's growth rate.
- It means that the running time will take at least this much, for large n.
- It represents best case performance.
- Example: In the Linear Search, the best case is when the element is found at the first index.

Time Complexity = Ω(1).

3) Big - Θ Notation (Θ):

- It gives the tight bound of an algorithms growth rate.
- The running time grows asymptotically at the same rate (both upper & lower bounded).
- It represents the average or typical performance.
- Example: Merge sort always divides & merges in $O(n \log n)$, regardless of input order.

So, $\Theta(n \log n)$.

* Reccurance Relation Using Binary Search

a (Array)	10	20	30	40	50	60	70
	1	2	3	4	5	6	7

i=1
j=7
x is the element we need to find

```
→ binary Search( a, i, j, x){
    mid = (i+j) /2 ;
    if(a[ mid ] == x) {
        return mid ; }
    else{
        if(a[mid] > x) {
            binary Search( a, i, mid-1, x) }
        else {
            binary Search (a, mid+1, j, x)}
        }
}
```

⟹ Recurrance Relation ⇒ $T(n) = T(n/2) + C$

C = Constant, n = number of input

* Recurrence Relation using Substitution Method

$$T(n) = \begin{cases} T(n/2) + c & ; \ n > 1 \\ 1 & ; \ n = 1 \end{cases}$$

$T(n) = T(n/2) + c$

$T(n/2) = T(n/4) + c$

$T(n/4) = T(n/8) + c$

Now

$T(n) = T(n/2) + c$

& $T(n/2) = T(n/4) + c$

replacing we get

$T(n) = T(n/4) + c + c$

$T(n) = T(n/4) + 2c$

$T(n) = T(n/8) + 3c$

$T(n) = T(n/2^4) + 4c$

⋮

repeating till k steps

$$T(n) = T(n/2^k) + k \cdot c$$

$n = 2^k$

$$T(n) = T(n/n) + k \cdot c$$

$$T(n) = T(1) + k \cdot c$$

$$T(n) = 1 + k \cdot c$$

$$\log_2 n = \log_2 2^k$$

$$\log_2 n = k$$

So, $\boxed{k = \log_2 n}$

Now, $T(n) = 1 + (\log_2 n) \cdot c$

So, $O(\log_2 n)$ is Time complexity of Binary Search using Back substitution

classmate
Date
Page 8

* Recurrance relation using substitution method

$$T(n) = \begin{cases} 1, & n = 1 \\ n * T(n-1); & n > 1 \end{cases}$$

$T(n) = n * T(n-1)$

$T(n-1) = (n-1) * T(n-2)$

$T(n-2) = (n-2) * T(n-3)$

⋮

~~k times~~

~~$T(n-k) = (n-k) * T(n-1-k)$~~

$T(n) = n * T(n-1)$

$T(n) = n * [(n-1) * T(n-2)]$

$T(n) = n(n-1) * T(n-2)$

$T(n) = n(n-1)(n-2) * T(n-3)$

$T(n) = n(n-1)(n-2)(n-3) * T(n-4)$

⋮

$(n-1)$ times

$$T(n) = n(n-1)(n-2)(n-3)\ldots(n-(n-1)) * T(n-(n-1))$$

$$T(n) = n(n-1)(n-2)(n-3)\ldots(1) * T(1)$$

$$T(n) = n! * T(1)$$

As, $T(1) = 1$

So, $T(n) = n!$

Time complexity $= O(n!)$

factorial growth.

Q) $$T(n) = \begin{cases} 1 ; & n = 1 \\ 2T\left(\frac{n}{2}\right) + n ; & n > 1 \end{cases}$$

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \frac{n}{2}$$

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + \frac{n}{4}$$

Now $T(n) = 2T\left(\frac{n}{2}\right) + n$

$$T(n) = 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}\right] + n$$

$$T(n) = 2^2 T\left(\frac{n}{2^2}\right) + 2n$$

$$T(n) = 2^2\left[2T\left(\frac{n}{8}\right) + \frac{n}{4}\right] + 2n$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + 3n$$

$\vdots$

i times

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + i \cdot n$$

$$\frac{n}{2^i} = 1$$

$$2^i = n$$

$$i = \log_2 n$$

$$T(n) = 2^{\log_2 n} \cdot T(1) + (\log_2 n)\, n$$

$$T(n) = n \cdot 1 + n \cdot \log n$$

$$T(n) = n \log n + n$$

Time Complexity is $O(n \log n)$

21 September 2025 — Sunday

① $$T(n) = \begin{cases} 1 & ; n = 1 \\ T(n-1) + \log n & ; n > 1 \end{cases}$$

$$T(n) = T(n-1) + \log n$$

$$T(n-1) = T(n-2) + \log(n-1)$$

$$T(n-2) = T(n-3) + \log(n-2)$$

$$T(n-3) = T(n-4) + \log(n-3)$$

$$T(n) = T(n-1) + \log n$$

$$T(n) = T(n-2) + \log(n-1) + \log n$$

$$T(n) = T(n-2) + \log(n \cdot (n-1))$$

$$T(n) = T(n-3) + \log(n-2) + \log((n-1) \cdot n)$$

$$T(n) = T(n-3) + \log(n \cdot (n-1) \cdot (n-2))$$

$$T(n) = T(n-4) + \log(n \cdot (n-1) \cdot (n-2) \cdot (n-3))$$

$$T(n) = T(n-i) + \log(n(n-1)(n-2)\cdots(n-(i-1)))$$

$n - i = 1$ $\quad i = n-1$

$n = i + 1$ $\quad (n - (n-1-1))$

$n - n + 1 + 1$

$$T(n) = T(n - (n-1)) + \log(n(n-1)(n-2)\cdots(n-(n-1-1)))$$

$$T(n) = T(1) + \log(n(n-1)\cdots(2))$$

$$T(n) = 1 + \log(n(n-1)\cdots(2)(1))$$

$$T(n) = 1 + \log(n!)$$

$$T(n) = 1 + \log n^n$$

$$T(n) = 1 + n \log n$$

$$TC = O(n \log n)$$

* Master Theorem

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

$a \geq 1$ & $b > 1$

Solution

$$T(n) = n^{\log_b a}\,[U(n)]$$

$U(n)$ depends upon $h(n)$

$$h(n) = \frac{f(n)}{n^{\log_b a}}$$

→ Relation between $h(n)$ & $U(n)$ is

	$h(n)$	$U(n)$
Case-1	$h(n) = n^r$; $r > 0$	$O(n^r)$
Case-2	$h(n) = n^r$, $r < 0$	$O(1)$
Case-3	$h(n) = (\log n)^i$; $i \geq 0$	$\frac{(\log_2 n)^{i+1}}{i+1}$

Q1) $T(n) = 8T\left(\frac{n}{2}\right) + n^2$

$a = 8$, $b = 2$, $f(n) = n^2$

$T(n) = n^{\log_b a} \cdot U(n)$

$T(n) = n^{\log_2 8} \cdot U(n)$

$T(n) = n^3 \cdot U(n)$

$U(n)$ depends upon $h(n)$

$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{n^2}{n^{\log_2 8}} = \frac{n^2}{n^3} = n^{-1}$

$U(n) = n^r$, $r < 0$

$= n^{-1}$, $r = -1 < 0$

so $U(n) = O(1) = 1$

$T(n) = n^3 \cdot U(n)$

$T(n) = n^3 \cdot 1$

$\boxed{T(n) = n^3}$

$TC = O(n^3)$

Q2) $T(n) = T\left(\frac{n}{2}\right) + c$

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

here, $a = 1$, $b = 2$, $f(n) = c$ (constant)

So $T(n) = n^{\log_b a} \cdot U(n)$

$T(n) = n^{\log_2 1} \cdot U(n)$

$T(n) = n^0 \cdot U(n)$ as $\log_2 1 = 0$

$T(n) = 1 \cdot U(n)$

$T(n) = U(n)$

Now $U(n)$ depends upon $h(n)$

Now $h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{c}{1} = c$

$h(n) = (\log_2 n)^0 \cdot c$

$i = 0$

So $U(n) = \frac{(\log_2 n)^{0+1}}{0+1} = \log_2 n$

Now $T(n) = \log_2 n$ so $TC = O(\log_2 n)$

Q3) $$T(n) = \begin{cases} T(\sqrt{n}) + \log n & \text{if } n \geq 2 \\ O(1) & \text{else} \end{cases}$$

$$T(n) = 1 \cdot T(n^{\frac{1}{2}}) + \log n \; ; \; n \geq 2$$

$a = 1$,

$$T(n) = T(\sqrt{n}) + \log n$$

$$T(n-1) = T(\sqrt{n-1}) + \log(n-1)$$

$$T(n-2) = T(\sqrt{n-2}) + \log(n-2)$$

$$T(n-3) = T(\sqrt{n-3}) + \log(n-3)$$

Q3) $$T(n) = 4\,T\left(\frac{n}{2}\right) + n$$

$a = 4$, $b = 2$ & $f(n) = n$

$$T(n) = n^{\log_b a} \cdot u(n)$$

$$T(n) = n^{\log_2 4} \cdot u(n)$$

$$T(n) = n^2 \cdot u(n)$$

$$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{n}{n^2} = n^{-1}$$

$u(n) = n^r$, $r < 0$; $O(1)$

$T(n) = n^2$ $\Longrightarrow$ $TC = O(n^2)$

Q4) $T(n) = 3T\left(\frac{n}{2}\right) + n^2$

$a = 3 \quad b = 2 \quad f(n) = n^2$

$$T(n) = n^{\log_b a} \cdot U(n)$$

$$T(n) = n^{\log_2 3} \cdot U(n)$$

$U(n)$ depends upon $h(n)$

$$h(n) = \frac{f(n)}{n^{\log_2 3}} = \frac{n^2}{n^{\log_2 3}}$$

$$h(n) = n^{(2 - \log_2 3)}$$

$r = 2 - \log_2 3 = 0.415 > 0$

$$h(n) = n^{(2 - \log_2 3)}$$

$$U(n) = n^{(2 - \log_2 3)}$$

$$T(n) = n^{\log_2 3} \cdot \frac{n^2}{n^{\log_2 3}}$$

$$T(n) = n^2$$

TC $\Rightarrow$ $\boxed{O(n) = n^2}$

Q5) $T(n) = 7\,T\left(\frac{n}{2}\right) + n^3$

$a = 7,\ b = 2,\ f(n) = n^3$

$T(n) = n^{\log_b a} \cdot U(n)$

$T(n) = n^{\log_2 7} \cdot U(n)$

$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{n^3}{n^{\log_2 7}}$

$r = 3 - \log_2 7 > 0$

~~$h(n) = n$~~ $h(n) = n^{3-\log_2 7}$

$T(n) = n^{\log_2 7} \cdot \frac{n^3}{n^{\log_2 7}}$

$T(n) = n^3$

$\boxed{TC = \Theta(n^3)}$

Q6) PT $T(n) = 1 + 2 + 3 + \cdots + n = \Theta(n^2)$

$T(n) = \sum_{i=1}^{n} i = \frac{n(n-1)}{2} = n^2\left(\frac{1}{2} - \frac{1}{n}\right)$

$T(n-1) = \frac{(n-1)(n-2)}{2}$

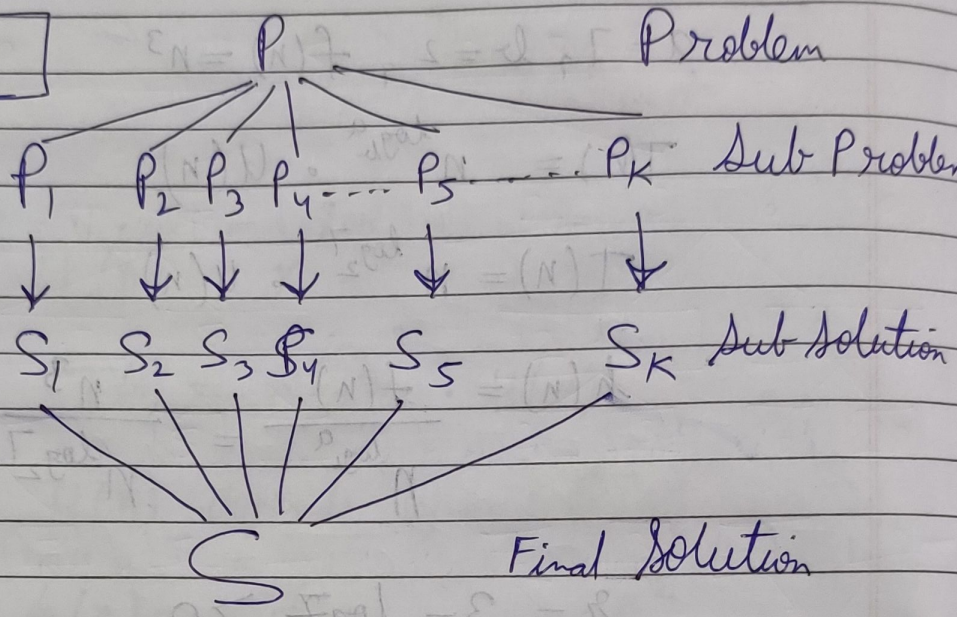
$T(n-2) = \frac{(n-2)(n-3)}{2}$

$\Theta(n^2)$

Unit – 2

Divide & Conquer Algorithm — 15%

D&C



- Recurrence Relations — Master's Theorem

D&C
- Binary Search
- Max-Min
- Merge sort
- Quick sort
- Matrix Multiplication

Sort & Search

1) How it works?

2) TC (Best, Average, Worst)

3) Recurrence Relation.

* Quick Sort (D&C)

35	50	15	25	80	20	90	45	+∞
V=	P						Q	

first element is pivot element i.e. V = 35

P & Q pointers

P pointer moves to RHS & it will stop at element > V

Q pointer moves to LHS & it will stop at element <= V

This happens because it selects elements greater than V on the RHS & less than V on the LHS.

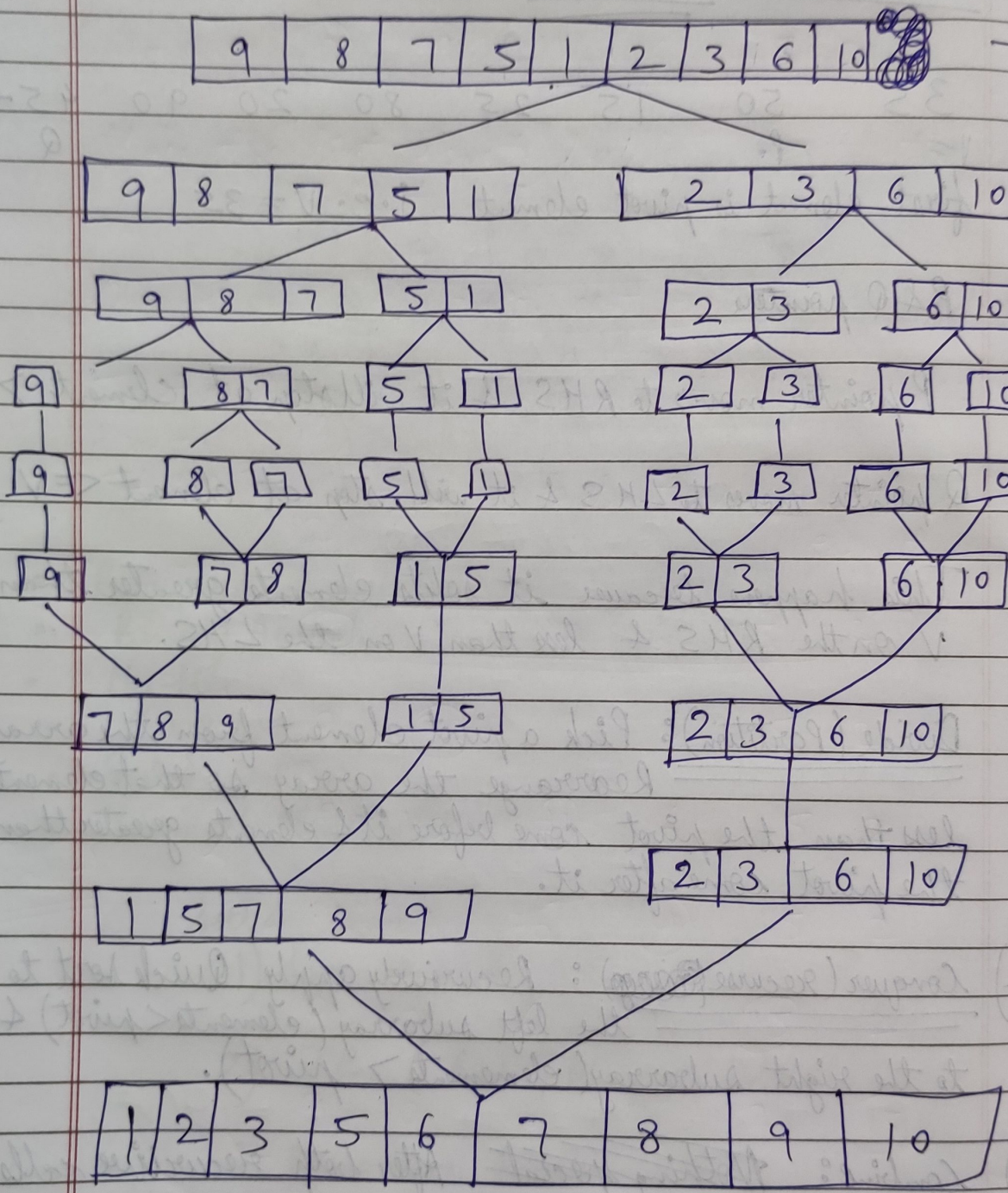
1) Divide (Partition): Pick a pivot element from the array. Rearrange the array so that elements less than the pivot come before it & elements greater than the pivot come after it.

2) Conquer (recurse): Recursively apply Quick sort to the left subarray (elements < pivot) & to the right subarray (elements > pivot).

3) Combine: ~~Nothing special~~ After both recursive calls finish the array is sorted.

Worst case ⇒ $T(n) = \Theta(n^2)$
Best Case ⇒ $T(n) = O(n \log n)$
Average Case ⇒ $T(n) = O(n \log n)$

Q) Merge Sort Algorithm: 9, 8, 7, 5, 1, 2, 3, 6, 10 — 4M



* Merge Sort Pseudo Code

```
MergeSort(A, p, r)
  if( p< r):
      q = (p+r) // 2
      MergeSort(A, p, q) #sort left part A[p...q]
      MergeSort(A, q+1, r)#sort right part A[q+1...r]
      Merge(A, p, q, r)# merge the two sorted halves
```

* Bubble Sort

10	9	11	6	15	2

Make the first element bubble i.e. 10 is the bubble element.

Compare bubble element with next element

If it is larger than bubble element shift bubble

It it is smaller than bubble element, swap with bubble element

Pass-1

(10)	9	11	6	15	2
9	(10)	11	6	15	2
9	10	(11)	6	15	2
9	10	6	(11)	15	2
9	10	6	11	(15)	2
9	10	6	11	2	(15)

After 1st pass the largest element is at the end of the array.

9	10	6	11	2	15

Pass-2

(9) 10 6 11 2

9 (10) 6 11 2

9 6 (10) 11 2

9 6 10 (11) 2

9 6 10 2 (11)

Now array becomes

9	6	10	2	11	15

Unsorted | Sorted

Pass-3

(9) 6 10 2

6 (9) 10 2

6 9 (10) 2

6 9 2 (10)

6	9	2	10	11	15

Unsorted | Sorted

Pass-4

(6) 9 2

6 (9) 2

6 2 (9)

6	2	9	10	11	15

Unsorted | Sorted

Pass-5

(6) 2

2 (6)

Complete Sorted Array ⟹

2	6	9	10	11	15

$n = 6$

number of passes = 5

Time Complexity = $O(n^2)$

* Insertion Sort

Insertion Sort is a simple, comparison based, stable sorting algorithm.

It works the same way we sort playing cards in our hands :-

1) Take one card at a time
2) Insert it into its correct position among the cards already sorted in your hand (the left part of the array)
3) Repeat until all cards are placed correctly.

$A = [\,5 \mid 3 \mid 8 \mid 6 \mid 2\,]$

Step-0 $A_{sorted} = [\,]$ Empty sorted array

Now insert elements from unsorted array A to A_{sorted} one by one

Step-1 Insert 5

$A_{sorted} = [\,5\,]$

Step-2 Insert 3, compare $3 < 5$, $3 < 5$ so place 3 on the left of 5.

$A_{sorted} = [\,3 \mid 5\,]$

Step-3 Insert 8, as 8 > 5, so 8 will be placed at the right of 5

A_{sorted} = | 3 | 5 | 8 |

Step-4 Insert 6, as 8 > 6 so we compare 6 & 5, 5 < 6, so 6 will be placed at left of 8 and right of 5

A_{sorted} = | 3 | 5 | 6 | 8 |

Step-5 Insert 2, as 2 < 8, 2 < 6, 2 < 5 & 2 < 3 so 2 will be placed at the left most position

A_{sorted} = | 2 | 3 | 5 | 6 | 8 |

Now The array is sorted completely.

* **Time Complexity**

- Best case is already sorted array

$$TC = O(n)$$

- Worst case is reverse sorted array

$$TC = O(n^2)$$

- Average case is roughly half of previous elements are shifted on each insertion

$$TC = O(n^2)$$

* Selection Sort

Selection Sort is a comparison based, in-place, unstable sorting algorithm.

How it works:

1) Repeatedly select the smallest element from the unsorted part of the array.

2) Place it at the beginning of the unsorted part.

3) Reduce the unsorted portion & repeat.

It's called "selection" because at every step we select the minimum element.

Unsorted Array A =

5	3	8	6	2

Step-1 min = 2, swap with first element i.e. 5

A =

2	3	8	6	5

Sorted | Unsorted

Step-2 min = 3 in unsorted array which is at its place

A =

2	3	8	6	5

Sorted | Unsorted

Step-3 min = 5 in unsorted array, swap with 8

A = | 2 | 3 | 5 | 6 | 8 |

sorted | unsorted

Step-4 min = 6 in unsorted array, its in place

final sorted array = | 2 | 3 | 5 | 6 | 8 |

* **Time Complexity**

1) Best Case is $O(n^2)$
2) Average Case is $O(n^2)$
3) Worst Case is $O(n^2)$

Radix Sort

It is a non-comparison based sorting algorithm

- It sorts digit by digit, starting from the least significant digit (LSD)
- It doesn't rely on comparing pairs of elements to sort but it uses a stable subroutine sorting algorithm to sort digits at each step.

* Its Time complexity can beat the lower bound of O(n log n) for comparison sorts.

Example

9	0	4
0	4	6
0	0	5
0	7	4
0	6	2
0	0	1

↓ hundred's ↓ ten's ↓ one's

In first step sort digits at one's place

Step-1

do not change relative position

0	0	1
0	6	2
9	0	4
0	7	4
0	0	5
0	4	6

ten's

Step-2 In second step sort digits at ten's place

⚠ Maintain the relative position

0	0	1
9	0	4
0	0	5
0	4	6
0	6	2
0	7	4

Step-3 In the 3rd step, sort digits at hundred's place

001
005
046
062
074
904

→ It is not comparison based

→ It is stable

→ Useful when sorting large integers, phone numbers, postal codes.

→ It requires extra space i.e. needs more arrays.

* Counting Sort

- Linear - Time Sorting algorithm used for sorting integers
- It works by counting the frequency of each distinct element & then using that information to place elements in the correct order.
- It is not comparison based, so it can beat the $O(n \log n)$ lower bound of comparison sorts.
- Given input size is "n" & range is "k"] Constraint

A =	4	2	2	3	4	1	2
	1	2	3	4	5	6	7

n = 7

k = range = 1 to 4 = 4

Create a separate array to count the frequency of each element of size equal to k

K (array)	
1	1
2	XX3
3	1
4	X2

⇒

1	1
2	3
3	1
4	2

↓ Value

↳ frequency (count of element in array A)

- It means that 1 is appearing once, 2 is appearing thrice, 3 is appearing once, four is appearing twice.

Sorted Array A =

1	2	2	2	3	4	4

✳ Time Complexity of Counting Sort

$TC = O(n+k)$

n = number of input

k = range

Space Complexity = $O(n+k)$

(Output Array + Counting Array)

✡ When to use?

- When elements are integers in a limited range like grades 0 to 100, ages 0 to 125.
- Not suitable if range of numbers (k) is very large compared to n.

for example if $n = 10^5$, $k = 200$ → very efficient

if $n = 10^5$, $k = 10^9$ → Not practical requires billions of extra ~~money~~ memory slots.

* Bucket Sort

- It is a distribution sort.
- Idea is to distribute the elements into a number of buckets, sort each bucket individually then concatenate all buckets to get the sorted array.
- It is useful when input is uniformly distributed over a known range like $[0, 1)$.

$A = [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]$

All numbers are in $[0, 1)$

Step-1 use 10 buckets : $B[0], B[1], \ldots, B[9]$

Each bucket corresponds to an interval of length 0.1

- $B[0] \rightarrow [0.0 - 0.1)$
- $B[1] \rightarrow [0.1 - 0.2)$
- $B[2] \rightarrow [0.2 - 0.3)$

Step-2 distribute elements into buckets

$0.78 \rightarrow B[7]$	$0.72 \rightarrow B[7]$	$0.23 \rightarrow B[2]$
$0.17 \rightarrow B[1]$	$0.94 \rightarrow B[9]$	$0.68 \rightarrow B[6]$
$0.39 \rightarrow B[3]$	$0.21 \rightarrow B[2]$	
$0.26 \rightarrow B[2]$	$0.12 \rightarrow B[1]$	

- Buckets after distribution:

B[0] = [ ]

B[1] = [0.17, 0.12]

B[2] = [0.26, 0.21, 0.23]

B[3] = [0.39]

B[6] = [0.68]

B[7] = [0.78, 0.72]

B[9] = [0.94]

* Step-3 Sort each bucket

B[1] = [0.12, 0.17]
B[2] = [0.21, 0.23, 0.26]

B[7] = [0.72, 0.78]

* Step-4 Concatenate buckets

Merge all buckets in order

[0.12, 0.17, 0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78, 0.94]

TC Best/Average case = O(n) Worst case = O(n log n)

classmate
Date
Page 37

22 September 2025 — Monday — 5:21 AM

* Greedy Algorithms

A greedy algorithm builds up a solution step by step, always choosing the option that looks best locally optimal at that moment, hoping it leads to the globally optimal solution.

General characteristics are :-

- **Greedy choice property:** A globally optimal solution can be achieved by making locally optimal (greedy) choices at each stage.
- **Optimal Substructure:** ~~A problem has an optimal substructure if an~~ An optimal solution to the problem contains within it optimal solutions to subproblems.
- **No Backtracking:** A greedy algorithm makes irreversible decisions. Once decision/choice is made, it is final, we don't go back & change it. This makes greedy faster but also riskier
- **Efficiency:** Greedy algorithms are often very fast & efficient, with many having a time complexity of $O(n \log n)$ or $O(n)$

Example: **Knapsack Problem** — Selecting items with the highest value-to-weight ratio to maximize the total value in a knapsack.

* Knapsack Problem (Fractional - greedy)

We have a knapsack with a maximum weight capacity W.

n items each with weight w_i & value v_i

- Goal is to choose items to maximize total value without exceeding weight capacity W.

Example

Objects	obj 1	obj 2	obj 3
Profit	25	24	15
Weight	18	15	10
P/W ratio	1.3	1.6	1.5

Knapsack capacity (M) = 20

Case-1 When greedy about Profit

obj 2	} 2
obj 1	} 18

$= 25 + \left(\frac{2}{15}\right) \times 24 = \boxed{28.2}$

(18 ← weights)

Case-2 When greedy about weight

$= 15 + \left(\frac{10}{15}\right) \times 24 = \boxed{31}$

obj 2	} 10
obj 3	} 10

Case-3 When greedy about profit/weight ratio

$= 24 + \left(\frac{5}{10}\right) \times 7.5$

$= 31.5$

Obj 3

Obj 2

5 + 15

Maximum in Case-3 when P/W is selected.

* Time Complexity

Computing Ratios = $O(n)$

Sorting by ratio = $O(n \cdot \log n)$

Iterating through items = $O(n)$

TC → Total = $O(n \log n)$

Space Complexity = $O(1)$

Q1) Knapsack capacity = 85 ——— 8M

Weight =	10	20	30	40	50	60
Value =	60	80	20	100	50	30
V/W =	6	4	0.66	2.5	1	0.5

value = $60 + 80 + 100 + \left(50 \times \frac{15}{50}\right)$

weight = $10 + 20 + 40 + (15) = 85$

Max Profit — Value = 240 + 15 = 255

TC = $O(n \log n)$; $n = 6$

Q2) Knapsack capacity $W = 100$ ——— 8M
number of items = 5

weight	50	40	30	20	10
Profit	1	2	3	4	5
P/W	0.02	0.05	0.10	0.20	0.50

weight = 10 + 20 + 30 + 40 = 100

profit = 5 + 4 + 3 + 2 = 14

TC = $O(n \log n)$; $n = 5$

Q3) $n = 4$, $M = 8$

P	3	5	6	10
W	2	3	4	5
P/W	1.5	1.6	1.5	2.0

Weight = 5 + 3 = 8

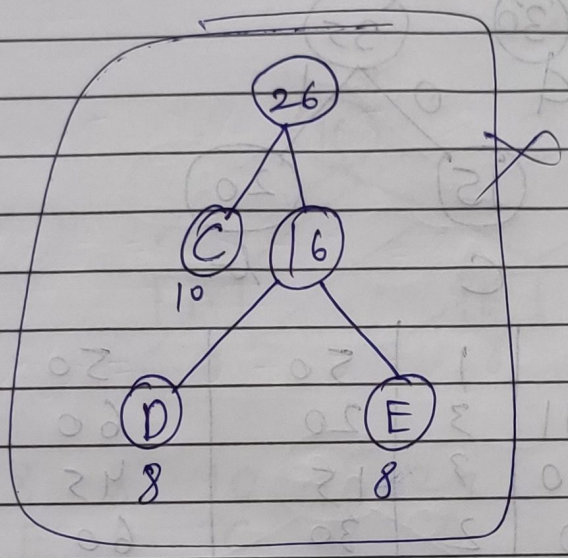
Profit = 10 + 5 = 15

* Huffman Coding — Greedy Technique

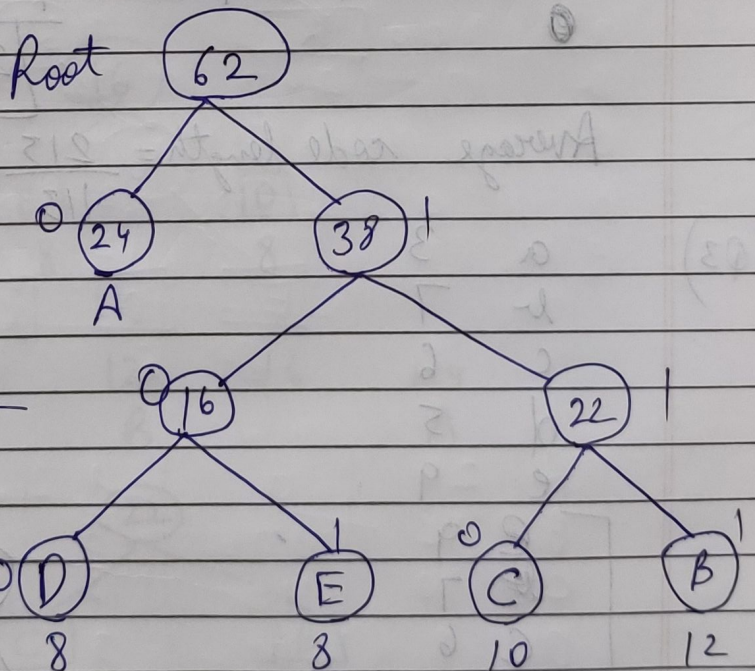
It is used to build an optimal prefix code that minimizes the expected code length.

Q1)

A	B	C	D	E	Letter
24	12	10	8	8	Frequency



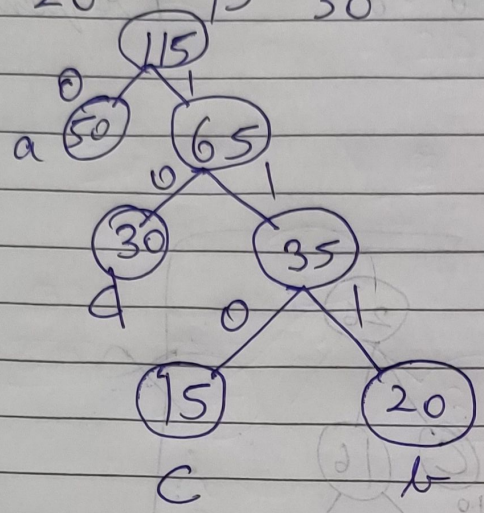
A	0	24	24
B	111	12	36
C	110	10	30
D	100	8	24
E	101	8	24
			138



Huffman code

Q2)

a	b	c	d
50	20	15	30



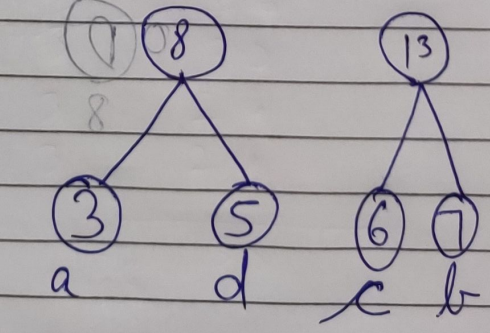
a	0	1	50	50
b	111	3	20	60
c	110	3	15	45
d	10	2	30	60
				215

Average code length $= \frac{215}{115} \approx 1.8696$ bits/symbol

Q3)

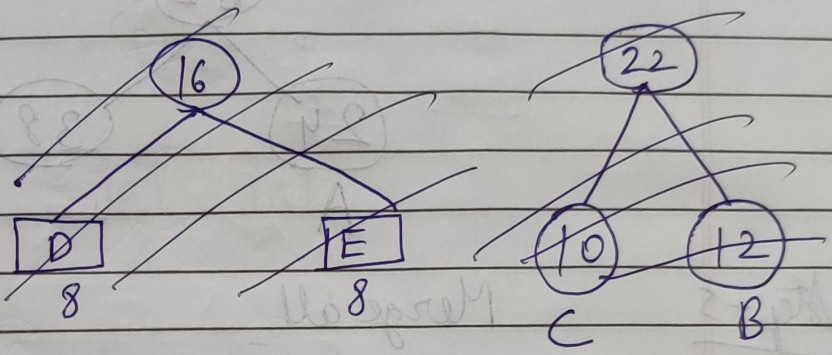
a	3
b	7
c	6
d	5
e	9

e	9
~~b~~	7
c	6
d	5
a	3



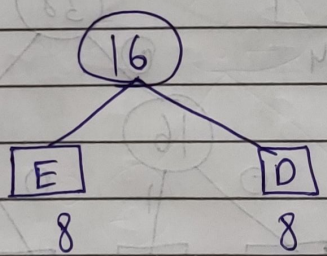
Q1)

A	B	C	D	E
24	12	10	8	8



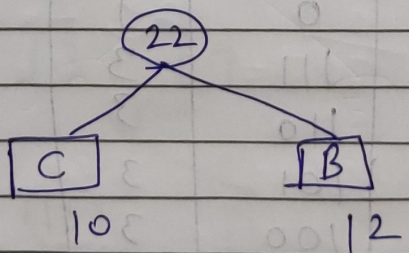
Step-1

8	8	10	12	24
E	D	C	B	A



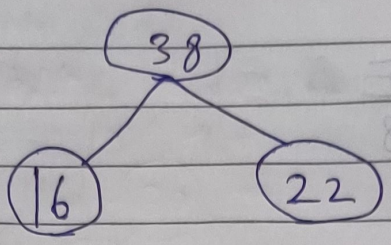
Step-2

10	12	16	24
C	B		A



Step-3

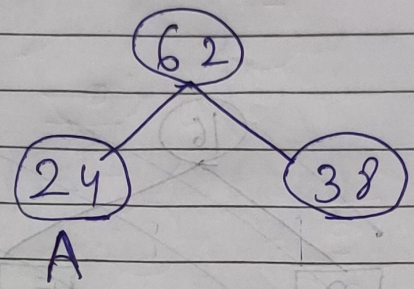
16	22	24
		A



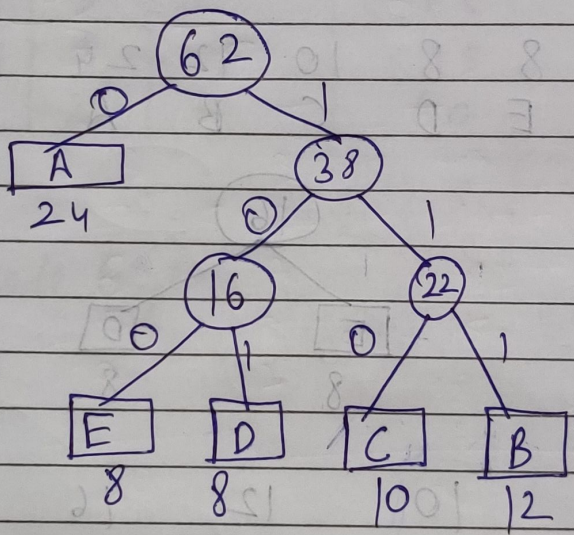
Step-4

24 38

A



Step-5 Merge all



A	24	0	1	24
B	12	111	3	36
C	10	110	3	30
D	8	101	3	24
E	8	100	3	24
	62			Total Bits 138

$$\frac{138}{62} \approx 2.22$$

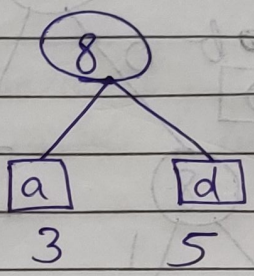
Q3)

a	b	c	d	e
3	7	6	5	9

Step -1

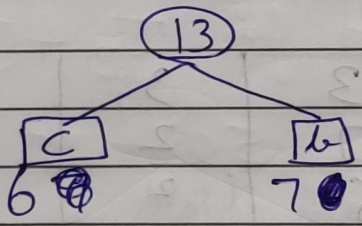
3	5	6	7	9
a	d	c	b	e

Step -1



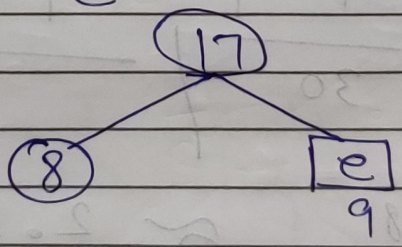
Step -2

6	7	8	9
c	b		e



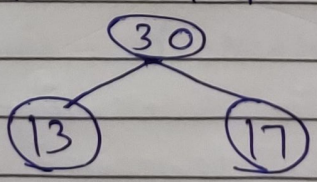
Step -3

8	9	13
	e	

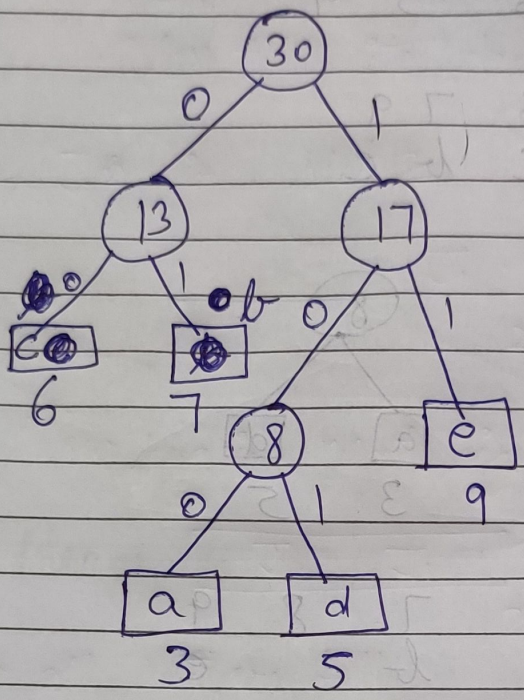


Step -4

13	17



Step-5 Merge all



a	100	3	3	9
b	01	7	2	14
c	00	6	2	12
d	101	5	3	15
e	11	9	2	18
		30		68

$$\frac{68}{30} \approx 2.267$$

* Job Sequencing (Greedy — Technique)

- **Problem Statement :** n jobs each has profit p_i & deadline d_i. Each job takes unit time. Schedule jobs, at most one per slot, to maximize total profit. Jobs are indivisible (0/1).

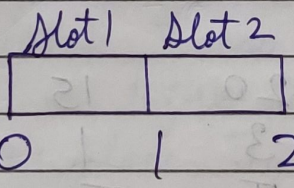
- Process jobs in decreasing profit order & place each job as late as possible before its deadline into a free slot.

- Time Complexity = $O(n \log n)$

Q1)

Jobs	J_1	J_2	J_3	J_4
Profit	50	15	10	25
Deadline	2	1	2	1

Step-1

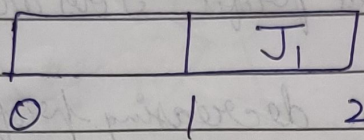


Jobs	J_1	J_4	J_2	J_3
Profit	50	25	15	10
deadline	2	1	1	2

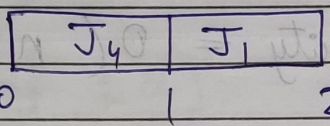
Sorting jobs in decreasing order.

Step -2 Placing maximum profit job in the latest available ~~slot~~ slot which is less than its deadline.

placing J_1 in slot 2



Step -3 Now placing next highest profit job i.e. J_4



Now all slots are full
So no space for J_3 & J_2

Profit = 50 + 25 = **75**

Q2)

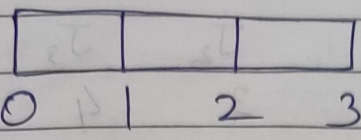
Profit	20	15	10	7	5	3
deadline	3	1	1	3	1	3
Job	J_1	J_2	J_3	J_4	J_5	J_6

number of jobs = 6

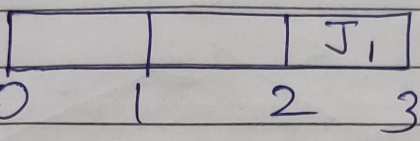
number of slots = 3

Step -1 Sort jobs in decreasing profit order already in decreasing order

Step-2 :



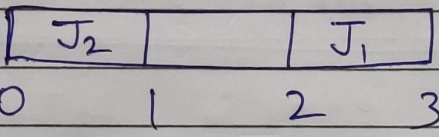
placing J_1 in slot-3 as it has highest profit & it has deadline=3



total profit = 20

Step-2

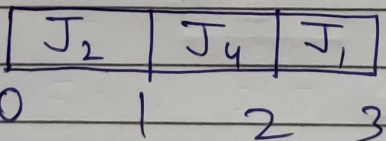
placing J_2 in slot-1



total profit = 35

Step-3 Now for J_3 deadline is 1 but slot-1 is occupied so we skip J_3

Step-4 for J_4 deadline is 3 so we can place it in 3rd slot or before slot-3. We have slot-2 empty. So we place J_4 in slot-2



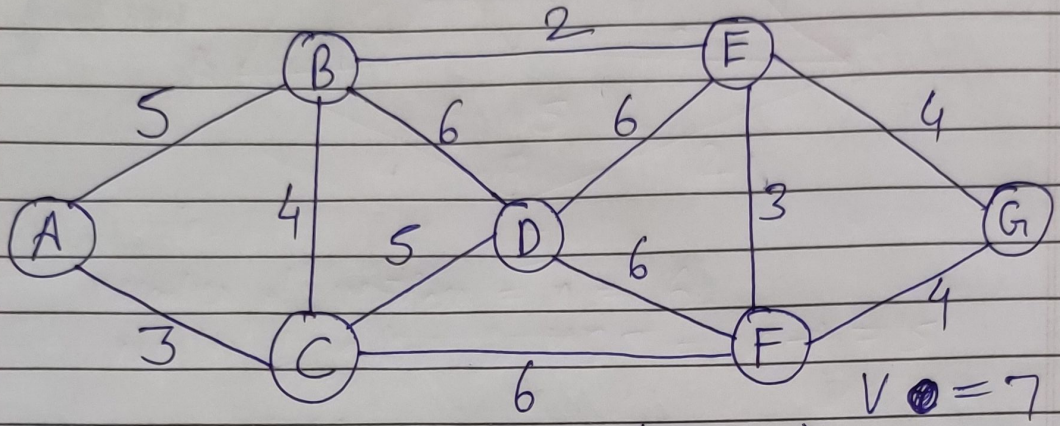
total profit = 42

Now all slots are full

Slot	Job	Profit
1	J_2	20
2	J_4	15
3	J_1	7
		42 total profit.

* Kruskal's algorithm — MST — Greedy Approach

Q1)



V = 7

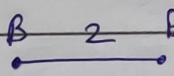
Minimum Spanning Tree (MST)

Step-1 arrange edges in sorted order

weight	2	3	3	4	4	4	5	5	6	6	6	6
edge	BE	AC	EF	BC	EG	FG	AB	CD	BD	DE	DF	CF

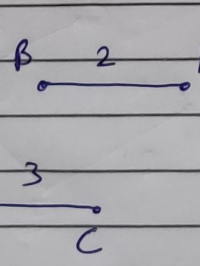
Step-2 Select minimum weight edge

2 — BE



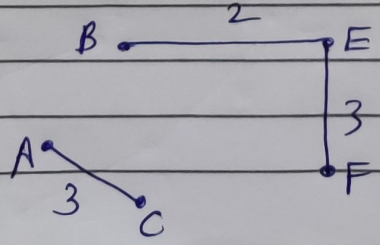
W = 2

Step-3 3 — AC



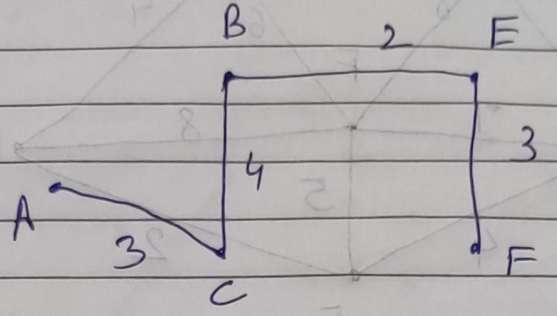
W = 5

Step-4 3-EF



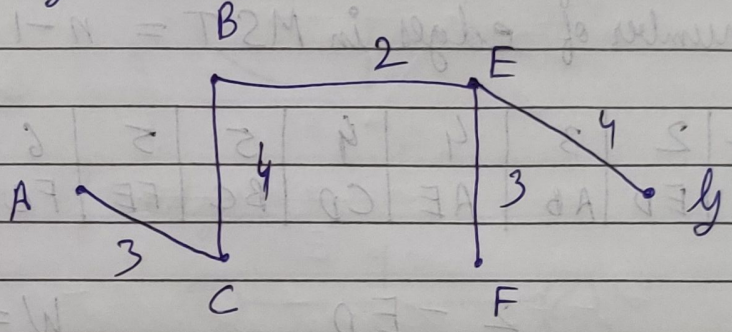
W = 8

Step-5 4 – BC



W = 12

Step-6 4 – Ey

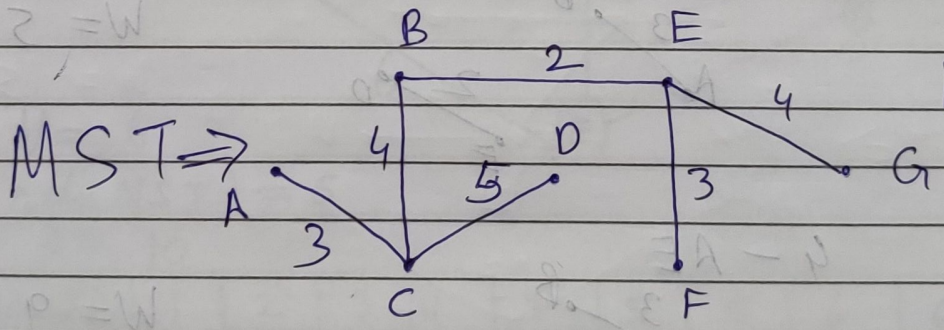


W = 16

Step-7 4 – Fy cannot be selected because it forms close loop.

5 – AB also same reason.

5 – CD ✓



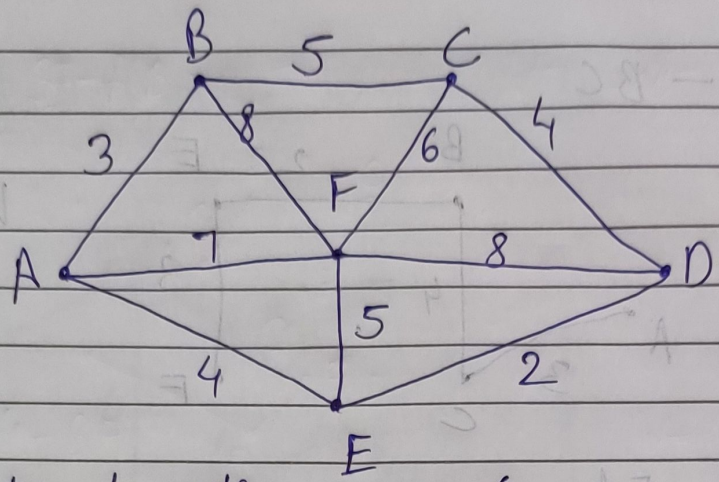
W = 21

number of vertices in graph is 7 = n

so number of edges in MST will be 6 (n-1)

minimum weight = 21

Q2)

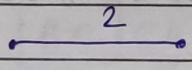


number of vertices = n = 6

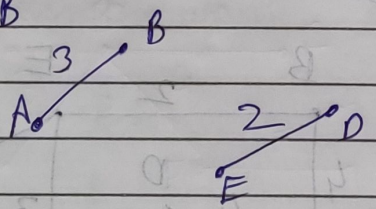
number of edges in MST = n − 1 = 5

weight	2	3	4	4	5	5	6	7	8	8
edge	ED	AB	AE	CD	BC	FE	FC	AF	BF	FD

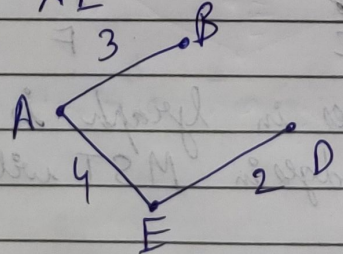
Step-1 2 − ED W = 2



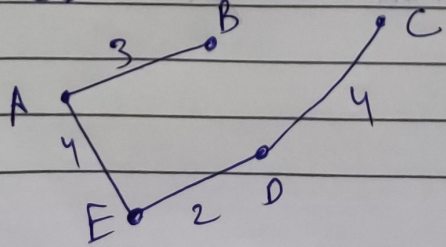
Step-2 3 − AB W = 5



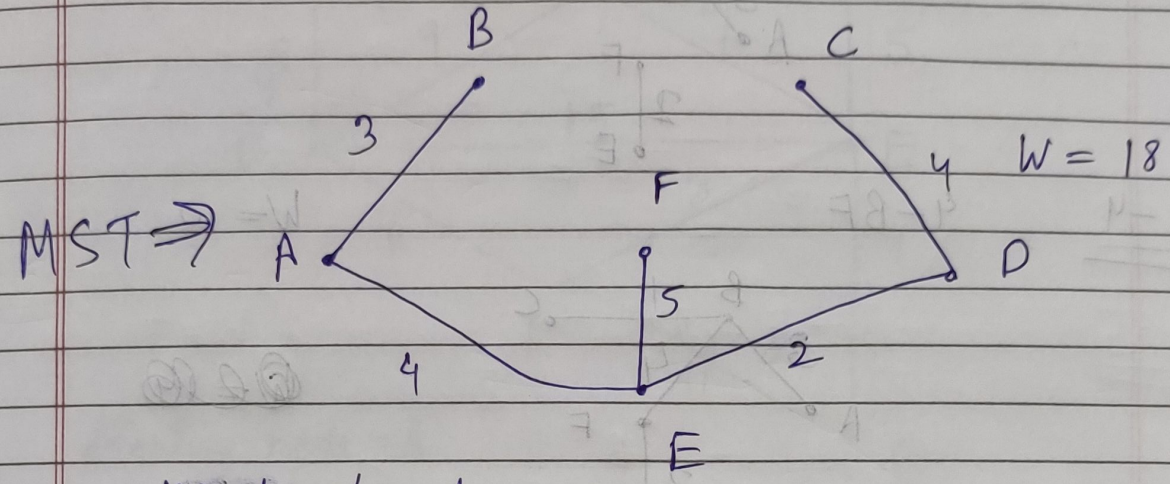
Step-3 4 − AE W = 9



Step-4 4 − CD W = 13

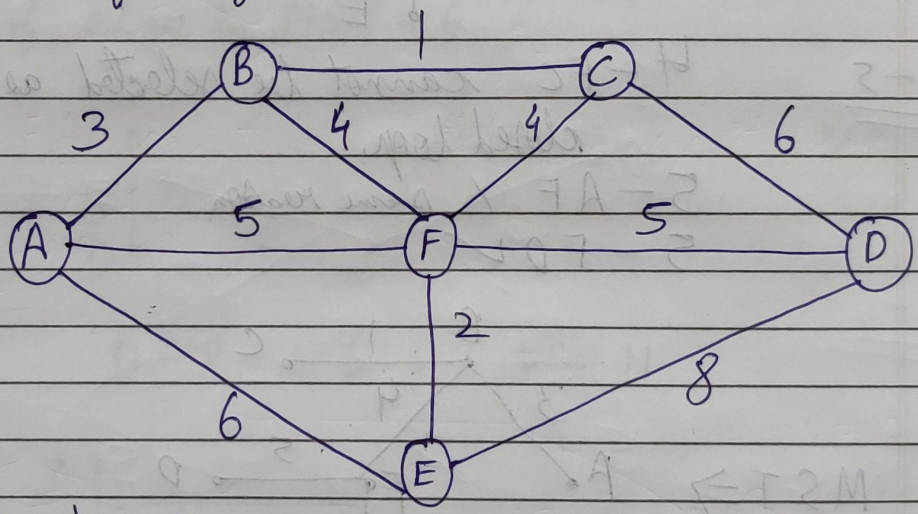


Step-5 5-BC cannot be selected
5-FE yes



number of edges = 5

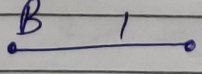
Q3)



V = 6
n (in MST) = 5
e = 10

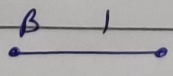
weight	1	2	3	4	4	5	5	6	6	8
edge	BC	FE	AB	BF	FC	AF	FD	AE	CD	ED

Step-1 1-BC



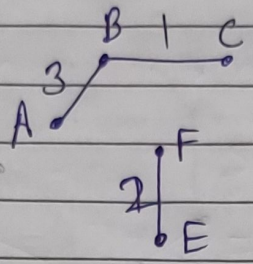
W = 1

Step-2 2 FE

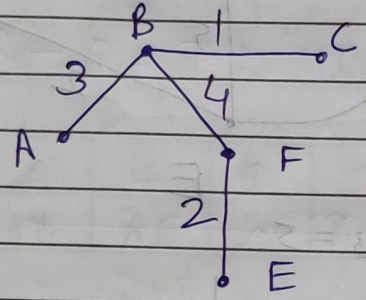


W = 3

Step-3 3-AB W = 6



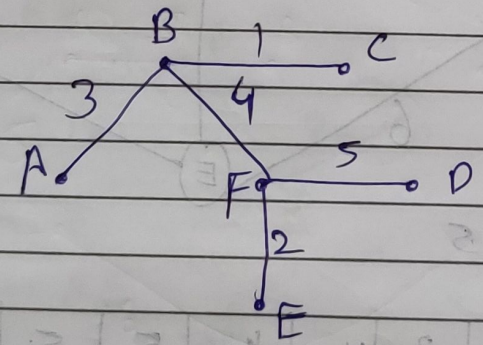
Step-4 4-BF W = 10



Step-5 4-FC cannot be selected as it forms a closed loop.

5-AF ✗ same reason

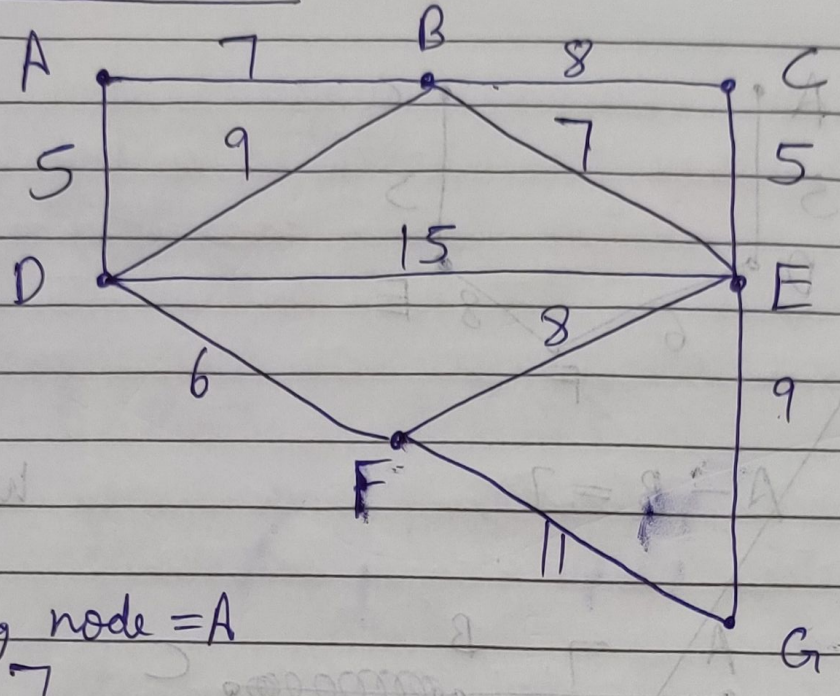
5-FD ✓



MST ⇒ W = 15

Number of edges = 5

* Prims Algorithm — MST — Greedy Approach

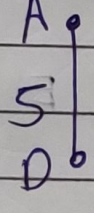


Starting node = A

V = 7

number of edges in MST = 6

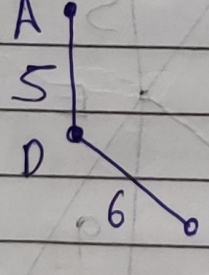
Step-1



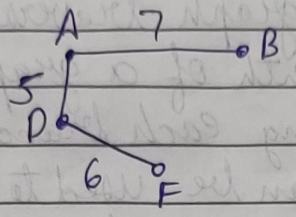
A - D = 5 W = 5

Step-2

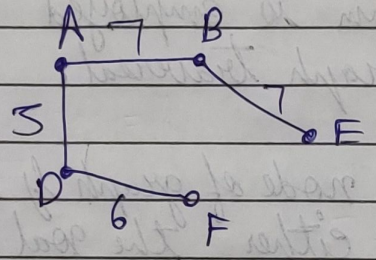
D - F = 6 W = 11



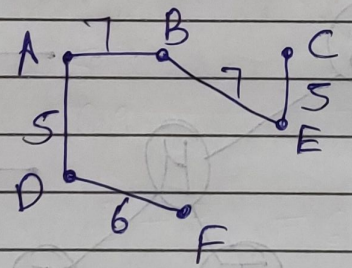
Step-3 A-B=7 W=18



Step-4 B-E=7 W= 25

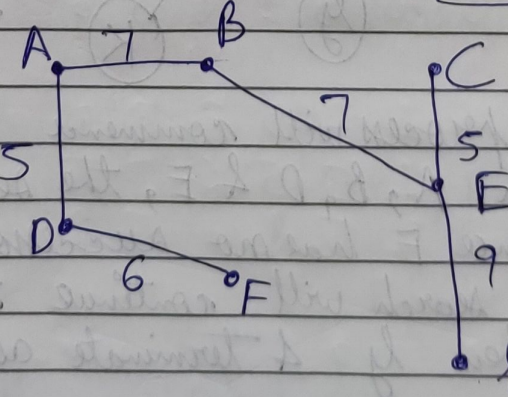


Step-5 E-C=5 W=30



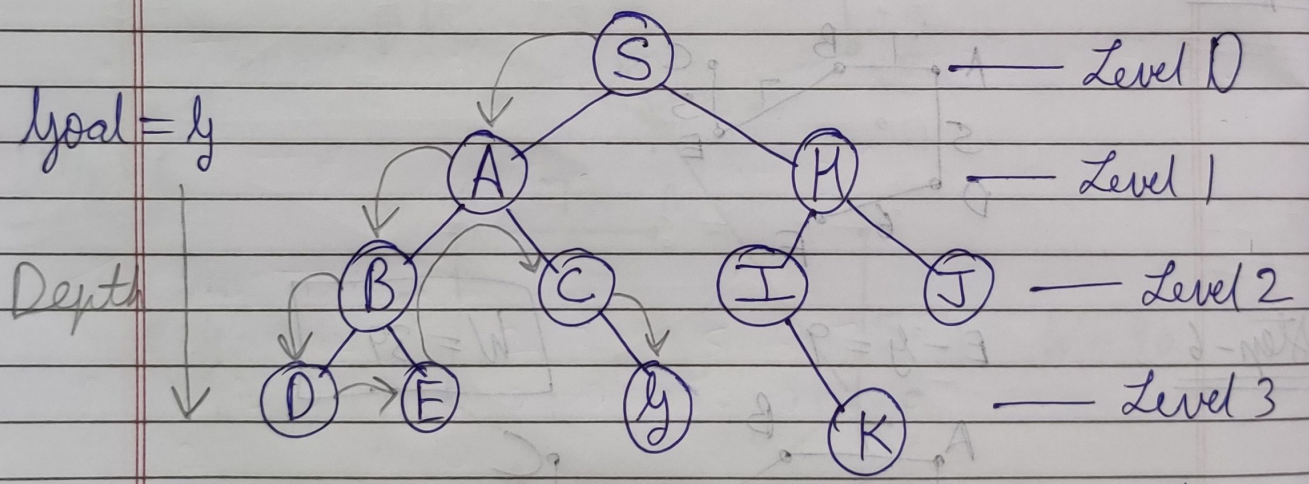
Step-6 E-y=9 W=39

MST⇒



* DFS (Depth First Search)

- It is a common graph traversal algorithm that explores the depth of a graph i.e. it goes as far as possible along each branch before backtracking. It can be used to search for paths, cycles or connected graphs within a graph.
- A Recursive algorithm is employed to explore all vertices in a graph traversal.
- Starting from initial node of graph G, DFS algorithm delves deeper until either the goal node or a node with no children is encountered.



- The search process will commence from root node S & follow path A, B, D & E, the search will backtrack since E has no successors. After backtracking, search will continue to traverse node C, then G & terminate as it reached goal.

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

1 October 2025 — Wednesday.

* Dynamic Programming

- It is a problem solving technique used to solve complex problems by breaking them into overlapping subproblems & solving each subproblem only once.
- It is especially useful for optimization problems like finding max/min, shortest path, max profit
- Optimal solution of the whole problem depends on the optimal solution of its subproblems.
- Example: Fibonacci Series

$$F(n) = F(n-1) + F(n-2);$$ with base $F(0)=0$ & $F(1)=1$

→ Without DP (simple recursion) ⇒ $O(2^n)$ exponential

→ with DP (tabulation) ⇒

```
F[0] = 0, F[1] = 1
for i = 2 to n:
    F[i] = F[i-1] + F[i-2]
```

Runs in $O(n)$ time.

* 0/1 Knapsack using DP

$m = 8$
$n = 4$

$P = \{1, 2, 5, 6\}$

$W = \{2, 3, 4, 5\}$

$x_i = 0/1$, $x = \{\quad\}$

- Sum of profits must be maximized

  $\max \sum p_i x_i$

- $\sum w_i x_i \leq m$, sum of weights less than or equal to capacity of bag

- Try all possible solutions & pick up the best one.

  Total solutions possible $= 2^n$

* Using Tabulation

P	W	i= \ w=0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3
5	4	3	0	0	1	2	5	5	6	7
6	5	4	0	0	1	2	5	6	6	7	8

max = 8 @ (2,3) + (6,5)

	x_1	x_2	x_3	x_4
{	0	1	0	1 }

P W

Q) W 2 1 3 2
V 12 10 20 15
W=5

	w	0	1	2	3	4	5
i = 0		0	0	0	0	0	0
W V							
2 12	1	0	0	12	12	12	12
1 10	2	0	10	12	22	22	22
3 20	3	0	10	12	22	30	32
2 15	4	0	10	15	25	30	37

Max profit = 37

@ (2,12), (2,15), (1,10).

Q) coins = {1, 3, 4}

amount = 7

dp[x] = minimum coins needed to make amount x.

Recurrence:

$$dp[x] = 1 + \min(dp[x-1], dp[x-3], dp[x-4])$$

base case ⇒ dp[0] = 0 → 0 coins needed to make 0.

- dp[0] = 0
- dp[1] = 1 + dp[0] = 1 → {1}
- dp[2] = 1 + dp[1] = 1 + 1 = 2 → {1, 1}
- dp[3] = 1 + dp[0] = 1 → {3}
- dp[4] = 1 + dp[0] = 1 → {4}
- dp[5] = 1 + min(dp[4], dp[2], dp[1]) = 1 + min(1, 2, 1) = 1 + 1 = 2 → {4, 1}
- dp[6] = 1 + min(dp[5], dp[3], dp[2]) = 1 + min(2, 1, 1) = 1 + 1 = 2 → {3, 3}
- dp[7] = 1 + min(dp[6], dp[4], dp[3]) = 1 + min(2, 1, 1) = 1 + 1 = 2 → {3, 4} or {4, 3}

Q) $d_1 = 1, d_2 = 4, d_3 = 6, x = 9$

Ans) $dp[x] = 1 + \min(dp[x-1], dp[x-4], dp[x-6])$

- $dp[0] = 0$
- $dp[1] = 1 + dp[0] = 1 \rightarrow \{1\}$
- $dp[2] = 1 + dp[1] = 2 \rightarrow \{1,1\}$
- $dp[3] = 1 + 2 = 3 \rightarrow \{1,1,1\}$
- $dp[4] = 1 + \min(dp[3], dp[0])$
  $= 1 + 0 = 1 \rightarrow \{4\}$
- $dp[5] = 1 + \min(dp[4], dp[1])$
  $= 1 + 1 = 2 \rightarrow \{1,4\}$
- $dp[6] = 1 + \min(dp[5], dp[2], dp[0])$
  $= 1 + 0 = 1 \rightarrow \{6\}$
- $dp[7] = 1 + \min(dp[6], dp[3], dp[1])$
  $= 1 + 1 = 2 \rightarrow \{1,6\}$
- $dp[8] = 1 + \min(dp[7], dp[4], dp[2]) = 2 \rightarrow \{4,4\}$
- $dp[9] = 1 + \min(dp[8], dp[5], dp[3])$
  $= 1 + 2 = 3 \rightarrow \{4,4,1\}$

* Principle of Optimality

- Principle of Optimality is the foundation of Dynamic Programming.
- If an optimal solution to the whole problem contains within it optimal solutions to its subproblems.
- If we break an optimal solution into smaller parts, each part itself be an optimal solution to that subproblem

* DP	Greedy
• Solves problems by breaking into subproblems & combining their optimal solutions	Makes a series of locally optimal choices hoping to reach a global optimum.
• Always gaurantees global optimum	May not always give global optimum.
• Considers all possibilities	Picks locally best at each step.
• Examples = 0/1 Knapsack Matrix Chain Multiplication Shortest Path.	Fractional Knapsack Huffman Coding.

Q) $T(n) = 7T\left(\frac{n}{2}\right) + n^3$ —— 3M

$a = 7 \quad b = 2 \quad f(n) = n^3$

$T(n) = n^{\log_b a} \cdot u(n) = n^{\log_2 7} \cdot u(n)$

$u(n)$ • depends upon $h(n)$.

$$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{n^3}{n^{\log_2 7}}$$

$$h(n) = n^{(3 - \log_2 7)}$$

$r = 3 - \log_2 7$

$r > 0$

~~$T(n) = O(n$~~

$$u(n) = \frac{n^3}{n^{\log_2 7}}$$

$$T(n) = n^{\log_2 7} \cdot \frac{n^3}{n^{\log_2 7}}$$

$$\boxed{T(n) = n^3}$$

* Longest Common Subsequence (LCS)

Q) X = R A J K O T length = 6

Y = R A I N C O A T length = 8

LCS = RAOT

Q) S T O N E

L O N G E S T

ONE , ST,

Q) P = (1,0,0,1,0,1,1,0,1,1,0,1)

Q = (0,1,1,0)

Q) P = X Y Z Y T X Y (7)

Q = Y T Z X Y X (6)

YTXY , YZXY (4)

ZXY

XYX

TXY

Q) ALGORITHM — 9

RHYTHM — 6

RTHM

Happy Dussehra ♡

Date Page 71

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

20 October 2025 — Thursday

Q1) What is Algorithm? Properties — 3M

Ans) An algorithm is a set of instructions which gives a step-by-step sequence of well-defined instructions for solving a problem or ~~perform~~ performing a computation.

* Properties :-

- Finiteness : An algorithm must always terminate after a finite number of steps.
- Definiteness : Each step must be clear & unambiguous.
- Input : An algorithm ~~must produce~~. Should have zero or more inputs.
- Output : An algo must produce at least one output i.e. solution of problem.
- Effectiveness : Each step of the algo must be simple, basic & executable.

Q) How can we analyze algo? Why analysis? — 3M

Ans) How to analyze:

- TC: Measures how much time an algo takes as input n increases.
- SC: Measures how much space i.e. memory (RAM/storage) an algo takes to run.
- Correctness: Does it give right output?
- Optimality: Is it most efficient Sol^n?
- Scalability: Can it handle large input sizes?

Why Analysis?

- To choose most efficient algor among many for same problem like Quick Sort vs Merge Sort.
- Helps us estimate running time & memory usage before implementation
- Ensures algo will work even for lagre inputs.
- Helps choose most efficient algo in terms of time & space

Qs) What is asymptotic notations? Explain — 3M

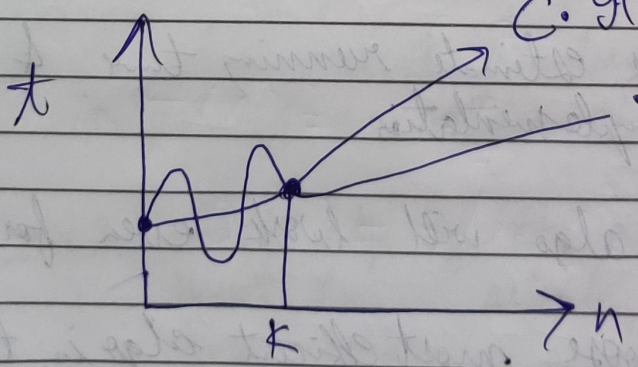
Ans3) Asymptotic notations are mathematical tools used to describe the efficiency of an algo in terms of time or space complexity as input size n becomes very large.

1) Big-O (O) → Upper Bound

- Describes worst case TC
- Represents max time an algo can take as input size n ↑.
- Example: If an algo takes at most $C \cdot n^2$ steps, we write

$$T(n) = O(n^2)$$

- It means algo will not take more than n^2 steps for large inputs.

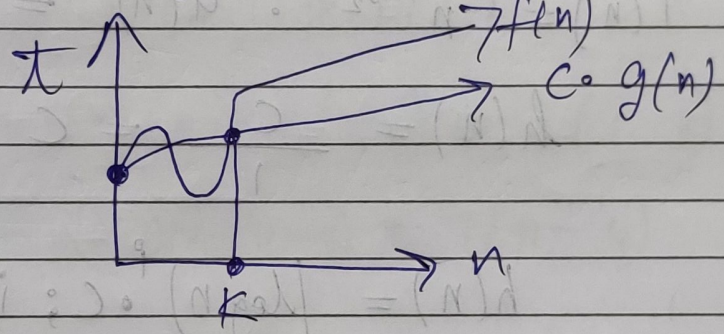


2) Omega (Ω) → Lower Bound

- Describes best case TC
- Represents minimum time an algo will take.
- Example: if an algo takes at best $C \cdot n \log n$ steps, we write

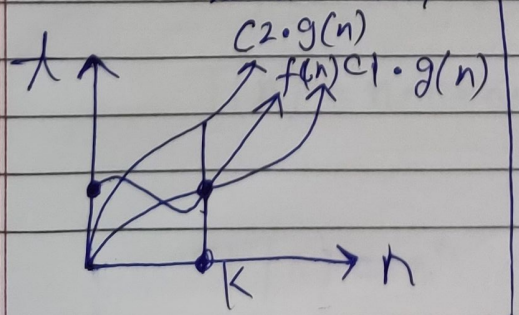
$$T(n) = \Omega(n \log n)$$

- Meaning the algo will take at least $n \log n$ steps. (not less)



3) Theta (θ) → Tight Bound

- Describes the average / exact bound.
- Used when an algorithm's TC is bounded both above & below by the same fn.
- Example: If an algorithm takes between $c1 \cdot n$ & $c2 \cdot n$ then $T(n) = \theta(n)$



Meaning the algorithm grows exactly proportional to n.

Q) $T(n) = T\left(\frac{n}{2}\right) + C$

$a = 1 \qquad b = 2 \qquad f(n) = C$

$$T(n) = n^{\log_b a} \cdot u(n)$$

$u(n)$ is dependent on $h(n)$

$$h(n) = \frac{f(n)}{n^{\log_b a}}$$

$$T(n) = n^{\log_2 1} \cdot u(n) = u(n)$$

$$h(n) = \frac{C}{1} = C$$

$$h(n) = (\log n)^i \cdot C; \quad i = 0$$

$$\text{So } u(n) = \frac{(\log_2 n)^{i+1}}{i+1}$$

$$u(n) = \log_2 n$$

$$\text{So } T(n) = \log_2 n$$

$$O(n) = O(\log_2 n)$$

Q) $T(n) = n * T(n-1)$; $n > 1$

$T(n-1) = (n-1) * T(n-2)$

$T(n-2) = (n-2) * T(n-3)$

$T(n-3) = (n-3) * T(n-4)$

$T(n) = n * T(n-1)$

$T(n) = n * (n-1) * T(n-2)$

$T(n) = n * (n-1) * (n-2) * T(n-3)$

$T(n) = n(n-1)(n-2)(n-3) * T(n-4)$

⋮

i times

$T(n) = n(n-1)(n-2)\cdots(n-i) * T(n-1-i)$

$n - i = 1$

$n = i + 1$

$T(n) = n(n-1)(n-2)\cdots 1 * T(i+1-1-i)$

$T(n) = n(n-1)(n-2)\cdots 1 * T(0)$

$T(n) = n \cdot n \cdot n \cdots n \cdot \left(1 - \frac{1}{n}\right)\left(1 - \frac{2}{n}\right)$

$\cdots\cdots (1)$

$T(n) = n^n$

Q) $T(n) = 2T\left(\frac{n}{2}\right) + n$

$a = 2 \quad b = 2 \quad f(n) = n$

$T(n) = n^{\log_2^2} \cdot U(n) = n \cdot U(n)$

$U(n)$ dependent on $h(n)$

$h(n) = \frac{f(n)}{n^{\log_b^a}} = \frac{n}{n} = 1$

$h(n) = 1 = \left(\log_2^n\right)^i \; ; \; i = 0$

So, $U(n) = \frac{\left(\log_2^n\right)^{i+1}}{i+1}$

$U(n) = \log_2^n$

$T(n) = n \cdot \log_2^n$

$O(n) = O\left(n \cdot \log_2^n\right)$

Q) $T(n) = T(n-1) + \log n$

$T(n-1) = T(n-2) + \log(n-1) \longrightarrow (1)$

$T(n-2) = T(n-3) + \log(n-2) \longrightarrow (2)$

$T(n-3) = T(n-4) + \log(n-3) \longrightarrow (3)$

$T(n) = T(n-1) + \log n$

$T(n) = T(n-2) + \log(n-1) + \log n$

$T(n) = T(n-3) + \log(n-2) + \log(n-1) + \log n$

$T(n) = T(n-4) + \log(n-3) + \log(n-2) + \log(n-1) + \log n$

$T(n) = T(n-i-1) + \log(n-i) + \ldots + \log(n)$

$n - i - 1 = 1$

$n = i + 2$

$i = n - 2$

$T(n) = T(1) + \log(n - n + 2) + \ldots + \log(n)$

$T(n) = 1 + \log n!$

$T(n) = 1 + \log n!$

$T(n) = O(n \log n)$

* Quick Sort

- Divide & Conquer Sorting Algo
- It works by selecting a pivot element, partioning array into 2 parts. Elements less than pivot in left part & elements greater than pivot in right part. & then recursively sorting the parts.

→ Best Case: $O(n \log n)$

→ Worst Case: $O(n^2)$

Q) Quick Sort

10, 20, 30, 50, [60], 80, 40, 70, 90, 100

Let pivot be 60

elements ~~before 60~~ on the left of 60 are less than 60
elements right of 60 are greater than 60.

10, 20, 30, 50, (40), [60], 80, 70, 90, 100

pivot = 40
10, 20, 30, 40, 50 ✓

pivot = 80
70, 80, 90, 100 ✓

Q) Merge Sort

9 8 7 5 | 1 2 3 6 10

9 8 | 7 5 | 1 2 3 | 6 10

9 | 8 | 7 | 5 | 1 | 2 | 3 | 6 | 10

9 8 | 7 5 | 1 2 | 3 | 6 10

8 9 | 5 7 | 1 2 | 3 | 6 10

5 7 8 9 | 1 2 3 6 10

1 2 3 5 6 7 8 9 10